

Лабораторная работа № 3

Задание: Из листа клетчатой бумаги размером $M \times N$ клеток удалили некоторые клетки. На сколько кусков распадется оставшаяся часть листа?

Пример. Если из шахматной доски удалить все клетки одного цвета, то оставшаяся часть распадется на 32 куска.

МОДИФИКАЦИЯ. То же, но перед удалением клеток лист склеили в цилиндр высотой N .

Каждая задача требует использовать некоторую структуру данных: стек, очередь и т.д.

Следует реализовать структуру данных 3 способами

А) через массив

Б) через связанный список

В) с использованием стандартной библиотеки языка (например, STL для C++)

Сравнить работоспособность и производительность каждой реализации.

Листинг №1:

```
#include <iostream>
#include <queue>
#include <chrono>
#include <windows.h>

using namespace std;
using namespace chrono;

const int MAX_M = 100;
const int MAX_N = 100;

struct Point
{
    int x;
    int y;
};

int original[MAX_M][MAX_N];
```

```
int M, N;
```

```
int dx[4] = { -1, 1, 0, 0 };
```

```
int dy[4] = { 0, 0, -1, 1 };
```

```
class ArrayQueue
```

```
{
```

```
    int max_size;
```

```
    Point* buffer;
```

```
    int head;
```

```
    int tail;
```

```
public:
```

```
    ArrayQueue(int size = 100000)
```

```
    {
```

```
        max_size = size;
```

```
        buffer = new Point[max_size];
```

```
        head = 0;
```

```
        tail = 0;
```

```
    }
```

```
    ~ArrayQueue()
```

```
    {
```

```
        delete[] buffer;
```

```
    }
```

```
    bool empty()
```

```
    {
```

```
        return head == tail;
```

```
    }
```

```
    int size()
```

```
    {
```

```
        if (head > tail)
```

```
            return max_size - head + tail;
```

```
        else
```

```
            return tail - head;
```

```
    }
```

```
    void push_back(Point p)
```

```
    {
```

```
        if (size() < max_size - 1)
```

```

    {
        buffer[tail] = p;
        tail = (tail + 1) % max_size;
    }
}

Point pop_front()
{
    Point tmp = buffer[head];
    head = (head + 1) % max_size;

    return tmp;
}
};

```

```

struct Node
{
    Point data;
    Node* next;

    Node(Point d, Node* n = NULL)
    {
        data = d;
        next = n;
    }
};

```

```

class ListQueue
{
    Node* head;
    Node* tail;

    int size_;

```

public:

```

    ListQueue()
    {
        head = NULL;
        tail = NULL;
        size_ = 0;
    }

    ~ListQueue()
    {

```

```

while (head)
{
    Node* tmp = head;
    head = head->next;
    delete tmp;
}

bool empty()
{
    return size_ == 0;
}

void push_back(Point p)
{
    if (tail == NULL)
    {
        tail = new Node(p);
        head = tail;
    }
    else
    {
        tail->next = new Node(p);
        tail = tail->next;
    }
    size_++;
}

Point pop_front()
{
    Point d = head->data;
    Node* tmp = head;
    head = head->next;
    delete tmp;

    if (head == NULL)
        tail = NULL;

    size_--;

    return d;
}
};

void copy_field(int dest[MAX_M][MAX_N])

```

```

{
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            dest[i][j] = original[i][j];
        }
    }
}

```

```

void bfs_array(int field[MAX_M][MAX_N], int sx, int sy)
{
    ArrayQueue q;
    field[sx][sy] = 2;
    q.push_back({ sx, sy });

    while (!q.empty())
    {
        Point p = q.pop_front();

        for (int k = 0; k < 4; k++)
        {
            int nx = p.x + dx[k];
            int ny = p.y + dy[k];

            if (nx >= 0 && nx < M &&
                ny >= 0 && ny < N)
            {
                if (field[nx][ny] == 1)
                {
                    field[nx][ny] = 2;

                    q.push_back({ nx, ny });
                }
            }
        }
    }
}

```

```

void bfs_list(int field[MAX_M][MAX_N], int sx, int sy)
{
    ListQueue q;
    field[sx][sy] = 2;
    q.push_back({ sx, sy });
}

```

```

while (!q.empty())
{
    Point p = q.pop_front();

    for (int k = 0; k < 4; k++)
    {
        int nx = p.x + dx[k];
        int ny = p.y + dy[k];

        if (nx >= 0 && nx < M &&
            ny >= 0 && ny < N)
        {
            if (field[nx][ny] == 1)
            {
                field[nx][ny] = 2;

                q.push_back({ nx, ny });
            }
        }
    }
}

void bfs_stl(int field[MAX_M][MAX_N], int sx, int sy)
{
    queue<Point> q;
    field[sx][sy] = 2;
    q.push({ sx, sy });

    while (!q.empty())
    {
        Point p = q.front();
        q.pop();

        for (int k = 0; k < 4; k++)
        {
            int nx = p.x + dx[k];
            int ny = p.y + dy[k];

            if (nx >= 0 && nx < M &&
                ny >= 0 && ny < N)
            {
                if (field[nx][ny] == 1)
                {
                    field[nx][ny] = 2;

```

```

        q.push({ nx, ny });
    }
}
}
}
}

```

```

int count_array()
{
    int field[MAX_M][MAX_N];
    copy_field(field);
    int pieces = 0;

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (field[i][j] == 1)
            {
                bfs_array(field, i, j);
                pieces++;
            }
        }
    }
    return pieces;
}

```

```

int count_list()
{
    int field[MAX_M][MAX_N];
    copy_field(field);
    int pieces = 0;

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (field[i][j] == 1)
            {
                bfs_list(field, i, j);
                pieces++;
            }
        }
    }
}

```

```

    return pieces;
}

int count_stl()
{
    int field[MAX_M][MAX_N];
    copy_field(field);
    int pieces = 0;

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (field[i][j] == 1)
            {
                bfs_stl(field, i, j);
                pieces++;
            }
        }
    }
    return pieces;
}

int main(){
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    cout << "Введите размеры поля M и N:\n";
    cin >> M >> N;

    cout << "\nВведите поле:\n";
    cout << "1 - клетка существует\n";
    cout << "0 - клетка удалена\n\n";

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            cin >> original[i][j];
        }
    }

    auto start1 = high_resolution_clock::now();
    int pieces1 = count_array();
    auto stop1 = high_resolution_clock::now();

```



```

auto duration1 = duration_cast<microseconds>(stop1 - start1);

auto start2 = high_resolution_clock::now();
int pieces2 = count_list();
auto stop2 = high_resolution_clock::now();
auto duration2 = duration_cast<microseconds>(stop2 - start2);

auto start3 = high_resolution_clock::now();
int pieces3 = count_stl();
auto stop3 = high_resolution_clock::now();
auto duration3 = duration_cast<microseconds>(stop3 - start3);

cout << "\n===== РЕЗУЛЬТАТЫ =====\n";

cout << "\nОчередь на массиве:\n";
cout << "Количество кусков: " << pieces1 << endl;
cout << "Время выполнения: " << duration1.count() << " мкс\n";

cout << "\nОчередь на списке:\n";
cout << "Количество кусков: " << pieces2 << endl;
cout << "Время выполнения: " << duration2.count() << " мкс\n";

cout << "\nSTL-очередь:\n";
cout << "Количество кусков: " << pieces3 << endl;
cout << "Время выполнения: " << duration3.count() << " мкс\n";

printf("\nФИО: Павленко Кирилл Дмитриевич\n");
printf("Группа: 020303-АИСа-о25\n");

return 0;
}

```

Листинг №2:

```

#include <iostream>
#include <queue>
#include <chrono>
#include <windows.h>

using namespace std;
using namespace chrono;

const int MAX_M = 100;
const int MAX_N = 100;

struct Point

```

```

{
    int x;
    int y;
};

int original[MAX_M][MAX_N];

int M, N;

int dx[4] = { -1, 1, 0, 0 };
int dy[4] = { 0, 0, -1, 1 };

class ArrayQueue
{
    int max_size;
    Point* buffer;

    int head;
    int tail;

public:

    ArrayQueue(int size = 100000)
    {
        max_size = size;
        buffer = new Point[max_size];

        head = 0;
        tail = 0;
    }

    ~ArrayQueue()
    {
        delete[] buffer;
    }

    bool empty()
    {
        return head == tail;
    }

    int size()
    {
        if (head > tail)
            return max_size - head + tail;
    }

```

```

        else
            return tail - head;
    }

    void push_back(Point p)
    {
        if (size() < max_size - 1)
        {
            buffer[tail] = p;
            tail = (tail + 1) % max_size;
        }
    }

    Point pop_front()
    {
        Point tmp = buffer[head];
        head = (head + 1) % max_size;

        return tmp;
    }
};

struct Node
{
    Point data;
    Node* next;

    Node(Point d, Node* n = NULL)
    {
        data = d;
        next = n;
    }
};

class ListQueue
{
    Node* head;
    Node* tail;

    int size_;

public:

    ListQueue()
    {

```

```

    head = NULL;
    tail = NULL;
    size_ = 0;
}

~ListQueue()
{
    while (head)
    {
        Node* tmp = head;
        head = head->next;
        delete tmp;
    }
}

bool empty()
{
    return size_ == 0;
}

void push_back(Point p)
{
    if (tail == NULL)
    {
        tail = new Node(p);
        head = tail;
    }
    else
    {
        tail->next = new Node(p);
        tail = tail->next;
    }
    size_++;
}

Point pop_front()
{
    Point d = head->data;
    Node* tmp = head;
    head = head->next;
    delete tmp;

    if (head == NULL)
        tail = NULL;
}

```

```

        size--;

    return d;
}
};

void copy_field(int dest[MAX_M][MAX_N])
{
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            dest[i][j] = original[i][j];
        }
    }
}

void bfs_array(int field[MAX_M][MAX_N], int sx, int sy)
{
    ArrayQueue q;
    field[sx][sy] = 2;
    q.push_back({ sx, sy });

    while (!q.empty())
    {
        Point p = q.pop_front();

        for (int k = 0; k < 4; k++)
        {
            int nx = p.x + dx[k];
            int ny = p.y + dy[k];

            if (ny < 0)
                ny = N - 1;

            if (ny >= N)
                ny = 0;

            if (nx >= 0 && nx < M)
            {
                if (field[nx][ny] == 1)
                {
                    field[nx][ny] = 2;
                    q.push_back({ nx, ny });
                }
            }
        }
    }
}

```

```

    }
  }
}

```

```

void bfs_list(int field[MAX_M][MAX_N], int sx, int sy)

```

```

{
    ListQueue q;
    field[sx][sy] = 2;
    q.push_back({ sx, sy });

    while (!q.empty())
    {
        Point p = q.pop_front();

        for (int k = 0; k < 4; k++)
        {
            int nx = p.x + dx[k];
            int ny = p.y + dy[k];

            if (ny < 0)
                ny = N - 1;

            if (ny >= N)
                ny = 0;

            if (nx >= 0 && nx < M)
            {
                if (field[nx][ny] == 1)
                {
                    field[nx][ny] = 2;
                    q.push_back({ nx, ny });
                }
            }
        }
    }
}

```

```

void bfs_stl(int field[MAX_M][MAX_N], int sx, int sy)

```

```

{
    queue<Point> q;
    field[sx][sy] = 2;
    q.push({ sx, sy });

    while (!q.empty())

```

```

{
    Point p = q.front();
    q.pop();

    for (int k = 0; k < 4; k++)
    {
        int nx = p.x + dx[k];
        int ny = p.y + dy[k];

        if (ny < 0)
            ny = N - 1;

        if (ny >= N)
            ny = 0;

        if (nx >= 0 && nx < M)
        {
            if (field[nx][ny] == 1)
            {
                field[nx][ny] = 2;
                q.push({ nx, ny });
            }
        }
    }
}

```

```

int count_array()
{
    int field[MAX_M][MAX_N];
    copy_field(field);
    int pieces = 0;

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (field[i][j] == 1)
            {
                bfs_array(field, i, j);
                pieces++;
            }
        }
    }
    return pieces;
}

```

```
}
```

```
int count_list()
{
    int field[MAX_M][MAX_N];
    copy_field(field);
    int pieces = 0;

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (field[i][j] == 1)
            {
                bfs_list(field, i, j);
                pieces++;
            }
        }
    }
    return pieces;
}
```

```
int count_stl()
{
    int field[MAX_M][MAX_N];
    copy_field(field);
    int pieces = 0;

    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (field[i][j] == 1)
            {
                bfs_stl(field, i, j);
                pieces++;
            }
        }
    }
    return pieces;
}
```

```
int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
```



```

cout << "Модификация: лист склеен в цилиндр.\n";

cout << "Введите размеры поля М и N:\n";
cin >> M >> N;

cout << "\nВведите поле:\n";
cout << "1 - клетка существует\n";
cout << "0 - клетка удалена\n\n";

for (int i = 0; i < M; i++)
{
    for (int j = 0; j < N; j++)
    {
        cin >> original[i][j];
    }
}

auto start1 = high_resolution_clock::now();
int pieces1 = count_array();
auto stop1 = high_resolution_clock::now();
auto duration1 = duration_cast<microseconds>(stop1 - start1);

auto start2 = high_resolution_clock::now();
int pieces2 = count_list();
auto stop2 = high_resolution_clock::now();
auto duration2 = duration_cast<microseconds>(stop2 - start2);

auto start3 = high_resolution_clock::now();
int pieces3 = count_stl();
auto stop3 = high_resolution_clock::now();
auto duration3 = duration_cast<microseconds>(stop3 - start3);

cout << "\n===== РЕЗУЛЬТАТЫ =====\n";

cout << "\nОчередь на массиве:\n";
cout << "Количество кусков: " << pieces1 << endl;
cout << "Время выполнения: " << duration1.count() << " мкс\n";

cout << "\nОчередь на списке:\n";
cout << "Количество кусков: " << pieces2 << endl;
cout << "Время выполнения: " << duration2.count() << " мкс\n";

cout << "\nSTL-очередь:\n";
cout << "Количество кусков: " << pieces3 << endl;

```

```
cout << "Время выполнения: " << duration3.count() << " мкс\n";

printf("\nФИО: Павленко Кирилл Дмитриевич\n");
printf("Группа: 020303-АИСа-о25\n");

return 0;
}
```

Комментарии к коду:

Данная программа реализует задачу подсчёта количества связных областей (кусков) на клетчатом поле размером $M \times N$, где каждая клетка может быть либо существующей (1), либо удалённой (0). Для решения используется алгоритм поиска в ширину (BFS), который позволяет обойти все клетки одной компоненты связности и пометить их как посещённые. Обход выполняется тремя способами реализации очереди: на массиве, на связном списке и с использованием стандартной библиотеки STL, что позволяет сравнить эффективность различных структур данных при одинаковом алгоритме. При обнаружении не посещённой клетки запускается BFS, который распространяется по всем соседним клеткам (вверх, вниз, влево, вправо), принадлежащим той же области, тем самым “помечая” весь кусок. Каждый запуск BFS соответствует одной компоненте связности, поэтому количество запусков определяет общее число кусков. Для каждой реализации измеряется время выполнения и выводится в микросекундах, после чего результаты сравниваются по скорости и одинаковости вычисленного количества компонентов.

Второй код является тем же самым, но с модификацией. В модификации реализована цилиндрическая топология поля: левая и правая границы считаются соединёнными. Это достигается за счёт переноса координаты точки при выходе за границы. Благодаря этому BFS может переходить через границу массива по горизонтали, что меняет структуру связности компонентов.

Примеры выполнения работы:

```
Введите размеры поля М и N:
5 5

Введите поле:
1 - клетка существует
0 - клетка удалена

0 1 0 0 1
1 1 0 1 0
0 0 0 0 1
1 1 0 0 1
1 0 1 0 0

===== РЕЗУЛЬТАТЫ =====

Очередь на массиве:
Количество кусков: 6
Время выполнения: 1016 мкс

Очередь на списке:
Количество кусков: 6
Время выполнения: 13 мкс

STL-очередь:
Количество кусков: 6
Время выполнения: 65 мкс

ФИО: Павленко Кирилл Дмитриевич
Группа: 020303-АИСa-о25
```

Код №1

```
Модификация: лист склеен в цилиндр.
Введите размеры поля М и N:
5 5

Введите поле:
1 - клетка существует
0 - клетка удалена

1 0 0 0 1
0 1 0 0 1
0 1 1 0 0
1 0 0 0 1
0 1 0 0 1

===== РЕЗУЛЬТАТЫ =====

Очередь на массиве:
Количество кусков: 4
Время выполнения: 779 мкс

Очередь на списке:
Количество кусков: 4
Время выполнения: 10 мкс
```

```
STL-очередь:  
Количество кусков: 4  
Время выполнения: 46 мкс  
ФИО: Павленко Кирилл Дмитриевич  
Группа: 020303-АИСa-о25
```

Код №2 (модификация)

Вывод: В ходе выполнения работы была реализована задача подсчёта количества связных областей на клетчатом поле с использованием алгоритма поиска в ширину (BFS). Для хранения очереди были разработаны три реализации: на массиве, на связном списке и с использованием стандартной библиотеки C++. Также была рассмотрена модификация задачи с цилиндрической топологией поля, где левая и правая границы считаются соединёнными.

По результатам эксперимента было установлено, что все три реализации дают одинаковый результат по количеству компонентов связности, что подтверждает корректность алгоритма. Однако по времени выполнения наблюдаются различия: наиболее быстрой оказалась реализация очереди на связном списке, затем — реализация с использованием STL, и наибольшее время показала очередь на массиве.